

Lab 02 - Authoring Your First Pod Manifest

Lab id	Lab 02
Day	1 - Cloud Native Foundations & Kubernetes Core Concepts
Duration	45 minutes
Namespace	kcna-lab02
Mapping	LO1 Develop a Kubernetes architectural proof of concept A1 Develop an architectural proof of concept K1 Process for developing proof of concepts

1. Objective

By the end of this lab you will be able to:

- Write a Pod manifest from scratch and explain every field you typed.
- Use `kubectl explain` and `kubectl run --dry-run=client -o yaml` as **authoring**

tools rather than guessing YAML.

- Distinguish what you declared from what the API server defaulted and what the kubelet observed.
- Serve real content into a container from a ConfigMap volume.
- Explain precisely what `containerPort` does - and, more importantly, what it does **not** do.
- Recognise, reproduce and diagnose `ErrImagePull` / `ImagePullBackOff`.

2. Prerequisites

- Kubernetes v1.30+ cluster (single-node kind is fine) and `kubectl` on PATH.
- Lab 01 completed, or equivalent comfort with `kubectl get/describe`.

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kcna-qa-20260905-control-plane	Ready	control-plane	58m	v1.37.0

```
cd courseware/labs/lab-02-first-pod-manifest
```

3. Scenario

Meridian Freight Pte Ltd has approved the Kubernetes proof of concept from Lab 01. The first workload to be modelled is the smallest, least risky piece of the Depot Portal: the static status page that depot supervisors load before their shift. It shows the depot register (data/depots.csv) and a build banner.

Your job in this lab is to express that workload as a Pod - the smallest deployable unit in Kubernetes - and to be able to defend every line of the manifest at the architecture review. No Deployment yet; that comes on Day 2. A bare Pod first, so that you understand what a Deployment is actually creating.

4. Step-by-step procedure

Step 1 - Create the namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab02 created
```

Make it your default for the rest of the lab so you stop typing -n:

```
kubectl config set-context --current --namespace=kcna-lab02
```

```
Context "kind-kcna-qa" modified.
```

Every command below still shows -n kcna-lab02 explicitly, so the lab works

either way. Remember to set it back to default at cleanup.

Step 2 - Look at the dataset you are going to serve

```
head -n 4 data/depots.csv
```

```
depot_code,depot_name,region,bays,cold_chain,shift_pattern,last_audit
MF-SIN-01,Jurong Port Depot,Singapore West,24,yes,3x8,2026-07-14
MF-SIN-02,Tuas Megahub,Singapore West,48,yes,3x8,2026-08-02
MF-SIN-03,Changi Air Freight Annex,Singapore East,16,no,2x12,2026-06-28
```

```
grep -c '^MF-' data/depots.csv
```

```
10
```

Step 3 - Turn the dataset into a ConfigMap

nginx serves whatever is in /usr/share/nginx/html. We will project two files there. Preview the object first:

```
kubectl -n kcna-lab02 create configmap depot-content \
  --from-file=index.html=data/depot-status.html \
  --from-file=depots.csv=data/depots.csv \
  --dry-run=client -o yaml | head -n 8
```

```
apiVersion: v1
data:
  depots.csv: |
    depot_code,depot_name,region,bays,cold_chain,shift_pattern,last_audit
    MF-SIN-01,Jurong Port Depot,Singapore West,24,yes,3x8,2026-07-14
    MF-SIN-02,Tuas Megahub,Singapore West,48,yes,3x8,2026-08-02
    MF-SIN-03,Changi Air Freight Annex,Singapore East,16,no,2x12,2026-06-28
    MF-SIN-04,Pasir Panjang Consolidation,Singapore South,32,yes,3x8,2026-08-19
```

--from-file=<key>=<path> lets you rename the key. Without the index.html= prefix the key would have been depot-status.html, and nginx would return 403 because there would be no index file. Create it for real:

```
kubectl -n kcna-lab02 create configmap depot-content \
  --from-file=index.html=data/depot-status.html \
  --from-file=depots.csv=data/depots.csv
```

```
configmap/depot-content created
```

```
kubectl -n kcna-lab02 get configmap depot-content \
  -o go-template='{{range $k,$v := .data}}{{ $k }}{{ "\n" }}{{end}}'
```

```
depots.csv
index.html
```

Step 4 - Ask the API what a Pod may contain

Do not open a browser. Ask the cluster:

```
kubectl explain pod
```

```
GROUP:
KIND:      Pod
VERSION:   v1

DESCRIPTION:
  Pod is a collection of containers that can run on a host. This resource is
  created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
  kind          <string>
  metadata      <ObjectMeta>
  spec          <PodSpec>
  status        <PodStatus>
```

Those five fields are universal. status is written by the control plane - you never author it.

```
kubectl explain pod.spec.containers | head -n 24
```

```

GROUP:
KIND:      Pod
VERSION:   v1

FIELD: containers <[]Container>

DESCRIPTION:
    List of containers belonging to the pod. Containers cannot currently be
    added or removed. There must be at least one container in a Pod. Cannot be
    updated.
    Container represents a single container that is expected to be run within a
    pod.
FIELDS:
  args <[]string>
  command <[]string>
  env <[]EnvVar>
  envFrom <[]EnvFromSource>
  image <string>
  imagePullPolicy <string>
  lifecycle <Lifecycle>
  livenessProbe <Probe>
  name <string>
  ports <[]ContainerPort>

```

Note <[]Container> - a list. Two of these fields are required:

```
kubectl explain pod.spec.containers.name
```

```

GROUP:
KIND:      Pod
VERSION:   v1

FIELD: name <string> -required-

DESCRIPTION:
    Name of the container specified as a DNS_LABEL. Each container in a pod must
    have a unique name (DNS_LABEL). Cannot be updated.

```

And the field that will bite you in section 6:

```
kubectl explain pod.spec.containers.ports.containerPort
```

```

GROUP:
KIND:      Pod
VERSION:   v1

FIELD: containerPort <integer> -required-

DESCRIPTION:
    Number of port to expose on the pod's IP address. This must be a valid port
    number, 0 < x < 65536.

```

Step 5 - Generate a skeleton, then finish it by hand

kubectl run with --dry-run=client never contacts the cluster to create anything; it renders the object the generator would have POSTed:

```
kubectl run depot-web --image=nginx:1.27-alpine --port=80 \
--dry-run=client -o yaml
```

```

apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: depot-web
  name: depot-web
spec:
  containers:
  - image: nginx:1.27-alpine
    name: depot-web
    ports:
    - containerPort: 80
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}

```

This is a starting point, not a deliverable. Three defects for a production PoC: resources: {}, a meaningless run: label, and no volume. Save it and fix it:

```

kubectl run depot-web --image=nginx:1.27-alpine --port=80 \
  --dry-run=client -o yaml > /tmp/my-depot-web.yaml

```

Now open /tmp/my-depot-web.yaml in your editor and make it look like this (type it; do not copy-paste if you can avoid it):

```

apiVersion: v1
kind: Pod
metadata:
  name: depot-web
  namespace: kcna-lab02
  labels:
    app.kubernetes.io/name: depot-web
    app.kubernetes.io/component: frontend
    app.kubernetes.io/part-of: depot-portal
    app.kubernetes.io/version: 0.1.0-poc
spec:
  containers:
  - name: web
    image: nginx:1.27-alpine
    imagePullPolicy: IfNotPresent
    ports:
    - name: http
      containerPort: 80
      protocol: TCP
    volumeMounts:
    - name: content
      mountPath: /usr/share/nginx/html
      readOnly: true
    resources:
      requests:
        cpu: 50m
        memory: 64Mi
      limits:
        cpu: 200m
        memory: 128Mi
  volumes:
  - name: content
    configMap:
      name: depot-content

```

Field-by-field, what you just wrote:

Field	Why it is there
apiVersion: v1	Pod lives in the core group, so no group prefix
metadata.namespace	pins the object to this lab; a manifest that relies on the ambient namespace is not reproducible
metadata.labels	the app.kubernetes.io/* set is the recommended label convention; selectors on Day 2 depend on them
containers[].name	required, DNS-label, unique in the Pod; it is what kubectl logs -c takes
image with a real tag	never :latest - see Lab 05 for why
imagePullPolicy: IfNotPresent	with a fixed tag there is nothing to re-pull
ports[].name: http	lets probes and Services reference the port by name
resources	without requests the scheduler is flying blind; without limits one Pod can starve a node
volumes[].configMap	projects the ConfigMap keys into the container filesystem as files

Step 6 - Validate before you apply

Client-side (schema only, no network):

```
kubectl apply -f /tmp/my-depot-web.yaml --dry-run=client
```

```
pod/depot-web created (dry run)
```

Server-side (full admission chain, defaulting and validation - but nothing is persisted):

```
kubectl apply -f /tmp/my-depot-web.yaml --dry-run=server
```

```
pod/depot-web created (server dry run)
```

--dry-run=server is the stronger check: it runs admission webhooks, quota and Pod Security Admission. Use it in CI.

Deliberately break it once to see validation fire. Change containerPort: 80 to containerPort: 80000 and re-run:

```
sed 's/containerPort: 80$/containerPort: 80000/' /tmp/my-depot-web.yaml \
| kubectl apply -f - --dry-run=server
```

```
The Pod "depot-web" is invalid: spec.containers[0].ports[0].containerPort: Invalid value:
80000: must be between 1 and 65535, inclusive
```

Note the field path spec.containers[0].ports[0].containerPort. Kubernetes validation errors always tell you exactly where to look.

Step 7 - Apply the reference manifest and inspect the result

```
kubectl apply -f manifests/20-depot-web-pod.yaml
```

```
pod/depot-web created
```

```
kubectl -n kcna-lab02 get pod depot-web -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED	NODE	READINESS	GATES			
depot-web	1/1	Running	0	18s	10.244.0.11	kcna-qa-20260905-control-plane
<none>		<none>				

Read READY 1/1: one container, one ready. Readiness comes from the readinessProbe in the reference manifest - httpGet /index.html on the port named http.

Now compare what you wrote with what the server stored:

```
kubectl -n kcna-lab02 get pod depot-web \
-o jsonpath='{.spec.dnsPolicy}{"\t"}{.spec.serviceAccountName}{"\t"}{.spec.schedulerName}{"\n"}'
```

```
ClusterFirst■default■default-scheduler
```

You wrote none of those. The API server defaulted them at admission time. The scheduler then wrote .spec.nodeName, and the kubelet wrote .status:

```
kubectl -n kcna-lab02 get pod depot-web \
-o jsonpath='{range .status.conditions[*]}{.type}={.status}{"\n"}{end}'
```

```
PodReadyToStartContainers=True
Initialized=True
Ready=True
ContainersReady=True
PodScheduled=True
```

Step 8 - Understand containerPort

containerPort publishes nothing and blocks nothing. Prove it. The reference manifest declares port 80 only. Reach a port that was never declared:

```
kubectl -n kcna-lab02 exec depot-web -c web -- \
sh -c 'wget -q0- http://127.0.0.1:80/index.html | head -n 2'
```

```
<!DOCTYPE html>
<html lang="en">
```

Now reach the Pod IP from another Pod, on a port you did declare, and note that nothing about the declaration was load-bearing - nginx opened 80 because its own configuration told it to, not because your YAML did:

```
POD_IP=$(kubectl -n kcna-lab02 get pod depot-web -o jsonpath='{.status.podIP}')
echo "$POD_IP"
```

```
10.244.0.11
```

```
kubectl -n kcna-lab02 run curl-probe --rm -it --restart=Never \
  --image=busybox:1.36 --command -- wget -qO- "http://${POD_IP}/depots.csv"
```

```
depot_code,depot_name,region,bays,cold_chain,shift_pattern,last_audit
MF-SIN-01,Jurong Port Depot,Singapore West,24,yes,3x8,2026-07-14
MF-SIN-02,Tuas Megahub,Singapore West,48,yes,3x8,2026-08-02
MF-SIN-03,Changi Air Freight Annex,Singapore East,16,no,2x12,2026-06-28
MF-SIN-04,Pasir Panjang Consolidation,Singapore South,32,yes,3x8,2026-08-19
MF-JHB-01,Senai Cross-Dock,Johor,20,no,2x12,2026-05-30
MF-JHB-02,Pasir Gudang Yard,Johor,28,yes,3x8,2026-07-21
MF-KUL-01,Port Klang North Gate,Selangor,36,yes,3x8,2026-08-11
MF-BKK-01,Laem Chabang Transit,Chonburi,18,no,2x12,2026-04-17
MF-HAN-01,Hai Phong Bonded Store,Hai Phong,22,yes,3x8,2026-06-09
MF-JKT-01,Tanjung Priok Staging,Jakarta,26,no,2x12,2026-07-03
pod "curl-probe" deleted
```

So what is containerPort for? Three real uses: it names a port so probes and Services can use port: http instead of a magic number; it feeds kubectl describe/tooling and dashboards; and kubectl port-forward and some service meshes read it. It is **documentation with a name attached**.

Step 9 - Reach the Pod from your workstation

```
kubectl -n kcna-lab02 port-forward pod/depot-web 8080:80
```

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

Leave that running. In a second terminal:

```
curl -s http://127.0.0.1:8080/ | grep STATUS
```

```
<p class="ok">STATUS: OK - served from a Pod in namespace kcna-lab02</p>
```

```
curl -s http://127.0.0.1:8080/depots.csv | wc -l
```

```
11
```

Return to the first terminal and press Ctrl+C to stop the forward.

```
Handling connection for 8080
Handling connection for 8080
^C
```

port-forward is a debugging tool. It tunnels over the API server, it is single-user and it dies with your terminal. It is not how you expose a service - that is Day 3.

5. Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script checks the namespace; that depot-content exists with both keys and that its depots.csv really carries the ten rows from data/depots.csv; that depot-web is Running, Ready, pinned to nginx:1.27-alpine, declares containerPort: 80, and has explicit requests and limits; that HTTP GETs for /index.html and /depots.csv return the seeded content; that server-side defaulting happened; and - if you have not cleaned it up yet - that depot-web-badtag is in ImagePullBackOff.

Expected tail:

```
Result: 14 passed, 0 failed
```

Full annotated evidence: [`verification/expected-output.md`](#).

6. Failure injection - one character, one broken deployment

manifests/90-depot-web-badtag.yaml is identical to a working Pod except for the image tag: nginx:1.27-alpin (the trailing e is missing). Apply it:

```
kubectl apply -f manifests/90-depot-web-badtag.yaml
```

```
pod/depot-web-badtag created
```

The API server accepted it. Schema validation cannot know which tags exist in a registry. Watch what happens next:

```
kubectl -n kcna-lab02 get pod depot-web-badtag
```

NAME	READY	STATUS	RESTARTS	AGE
depot-web-badtag	0/1	ErrImagePull	0	8s

Wait about 40 seconds and look again:

```
kubectl -n kcna-lab02 get pod depot-web-badtag
```

NAME	READY	STATUS	RESTARTS	AGE
depot-web-badtag	0/1	ImagePullBackOff	0	45s

kubectl logs cannot help - no container was ever created:

```
kubectl -n kcna-lab02 logs depot-web-badtag
```

```
Error from server (BadRequest): container "web" in pod "depot-web-badtag" is waiting to start: trying and failing to pull image
```

The evidence is in the events:

```
kubectl -n kcna-lab02 describe pod depot-web-badtag | tail -n 9
```

Events:				
Type	Reason	Age	From	Message
Normal	Scheduled	62s	default-scheduler	Successfully assigned kcna-lab02/depot-web-badtag to kcna-qa-20260905-control-plane
Normal	Pulling	22s (x3 over 62s)	kubelet	Pulling image "nginx:1.27-alpin"
Warning	Failed	21s (x3 over 61s)	kubelet	Failed to pull image "nginx:1.27-alpin": failed to pull and unpack image "docker.io/library/nginx:1.27-alpin": failed to resolve reference "docker.io/library/nginx:1.27-alpin": docker.io/library/nginx:1.27-alpin: not found
Warning	Failed	21s (x3 over 61s)	kubelet	Error: ErrImagePull
Normal	BackOff	8s (x4 over 60s)	kubelet	Back-off pulling image "nginx:1.27-alpin"
Warning	Failed	8s (x4 over 60s)	kubelet	Error: ImagePullBackOff

Diagnosis

Four things to read, in this order:

1. Scheduled succeeded. Not a scheduling problem - no taint, selector or resource issue.
2. Pulling image "nginx:1.27-alpin" - the kubelet is echoing the reference **you** asked for. Compare it character by character with what you meant. This is where the bug is visible.
3. ****failed to resolve reference "docker.io/library/nginx:1.27-alpin": ...: not found.*** Containerd expanded the short name to its fully-qualified form (docker.io/library/... - see Lab 05) and the registry returned a 404 for that tag. not found means the **tag** does not exist. Contrast with 401 Unauthorized, which would mean a credentials problem, and x509: certificate signed by unknown authority, which would mean a TLS/proxy problem.
4. ErrImagePull then ImagePullBackOff. These are two stages of the same thing. ErrImagePull is a single failed attempt. After repeated failures the kubelet switches to ImagePullBackOff and applies exponential back-off (10s, 20s, 40s ... capped at 5 minutes), which is why the status flips and the x3/x4 counters in the events climb slowly.

Note RESTARTS 0 throughout: nothing restarted, because nothing ever started.

Repair

A Pod's image field is mutable, so you can patch in place:

```
kubectl -n kcna-lab02 set image pod/depot-web-badtag web=nginx:1.27-alpine
```

```
pod/depot-web-badtag image updated
```

```
kubectl -n kcna-lab02 get pod depot-web-badtag -w
```

NAME	READY	STATUS	RESTARTS	AGE
depot-web-badtag	0/1	ImagePullBackOff	0	2m10s
depot-web-badtag	0/1	ContainerCreating	0	2m38s
depot-web-badtag	1/1	Running	0	2m40s

Press Ctrl+C. The kubelet noticed the spec change, resolved the correct tag and started the container - again without you deleting anything.

In real life you almost never patch a bare Pod: you fix the manifest in Git and

let a Deployment roll it out. That is Lab 07.

Remove the injected Pod when you are done (namespace-scoped, by name):

```
kubectl -n kcna-lab02 delete pod depot-web-badtag
```

```
pod "depot-web-badtag" deleted
```

7. Troubleshooting

Symptom	Likely cause	Fix
error: error validating "...": ... unknown field "spec.container"	Field name typo (singular/plural, wrong nesting level)	Check the exact path with kubectl explain pod.spec; re-run kubectl apply --dry-run=server before applying
The Pod "depot-web" is invalid: spec.containers[0].ports[0].containerPort: Invalid value: 80000	Value outside the schema's allowed range	Read the field path in the error; fix that exact field
ErrImagePull / ImagePullBackOff, event says not found	Tag or repository name is wrong	Correct the tag; verify with crictl images or by pulling on your workstation
ImagePullBackOff, event says 401 Unauthorized or pull access denied	Private registry with no imagePullSecrets	Create a kubernetes.io/dockerconfigjson Secret and reference it in spec.imagePullSecrets (Lab 15)
Pod Running but READY 0/1	readinessProbe failing - wrong path, port or the app is slow to start	kubectl describe pod → look for Readiness probe failed.; raise initialDelaySeconds or fix the path
nginx returns 403 Forbidden	ConfigMap mounted but no key named index.html	Recreate with --from-file=index.html=...; confirm with kubectl get cm depot-content -o yaml
Error from server (BadRequest): container "web" ... is waiting to start from kubectl logs	There is no container yet, so there is no log stream	Use kubectl describe pod and read the Events; logs only works once a container has been created
unable to forward port because pod is not running	port-forward started against a Pod that is not Running	Wait for kubectl wait --for=condition=Ready pod/depot-web, then retry
Pod stuck Pending with FailedScheduling ... Insufficient cpu	Your requests exceed the node's allocatable	Lower resources.requests; check kubectl describe node

8. Cleanup

Reset your default namespace first if you changed it in Step 1:

```
kubectl config set-context --current --namespace=default
```

```
Context "kind-kcna-qa" modified.
```

Then delete only this lab's namespace:

```
kubectl delete namespace kcna-lab02
```

```
namespace "kcna-lab02" deleted
```

9. What you learned

- A Pod manifest is four authored fields - apiVersion, kind, metadata, spec - plus a status you never write.
- kubectl explain and --dry-run=client -o yaml turn manifest authoring from recall into discovery. --dry-run=server additionally runs admission.
- Generators produce skeletons: no resources, no volumes, poor labels. Always finish the file by hand.
- The API server defaults fields (dnsPolicy, serviceAccountName, schedulerName, restartPolicy) at admission; the scheduler and kubelet then write more. kubectl get -o yaml is desired + defaulted + observed.
- containerPort is named documentation. It does not open, publish or restrict anything; the process inside the container decides what it listens on.
- A ConfigMap volume projects keys as files, and --from-file=<key>=<path> lets you control the filename.
- ErrImagePull is one failure; ImagePullBackOff is the back-off state after several. logs is useless before a container exists - describe is the tool.

10. Further reading

- [Pods](#)
- [Pod Lifecycle](#)
- [Kubernetes Objects - Understanding Kubernetes objects](#)
- [Recommended Labels](#)
- [Images and imagePullPolicy](#)
- [Configure a Pod to Use a ConfigMap](#)
- [Managing Resources for Containers](#)
- [Use Port Forwarding to Access Applications in a Cluster](#)
- [Debug Pods](#)